

Data Structures – Lab Assessment

Duration: 2 Hours 30 Minutes

Constraints: Object-Oriented Programming (Classes), standard library containers (`std::vector`, `std::list`, etc.), and doubly linked lists are strictly forbidden. All memory management must be handled manually using raw pointers, structs, pass-by-reference, `new`, and `delete`.

The Token Ring Allocator (Heap Management & Circular Basics)

Scenario: A legacy mainframe uses a Token Ring network topology where each hardware node passes a communication token to its immediate neighbor in a continuous loop. You must initialize this network and ensure proper memory allocation on the heap, as stack-allocated nodes would be destroyed the moment the initialization function returns.

Tasks:

- Define a struct `NetworkNode` containing an integer `nodeID` and a `NetworkNode*` `next` pointer.
- Write a standalone function `addNodeToRing(NetworkNode*& tail, int id)`. Notice you are maintaining the tail pointer rather than the head. When a new node is dynamically allocated on the heap, it should be inserted after the tail, the new node's next pointer should loop back to the head (which is `tail->next`), and the tail pointer should be updated to this new node.
- Write a standalone function `destroyRing(NetworkNode*& tail)`. Because this is a circular linked list on the heap, a naive traversal with `delete` will result in an infinite loop. You must explicitly break the cycle, safely deallocate every node to prevent memory leaks, and set the tail to null.

The Round-Robin OS Scheduler (Circular Traversal & Selective Deletion)

Scenario: You are writing a CPU scheduler for a lightweight operating system. Processes are stored in a single circular linked list. The CPU gives each process a fixed slice of execution time. If a process finishes its task, it must be removed from the circle so the CPU doesn't waste cycles polling an empty task.

Tasks:

- Define a struct `ProcessNode` containing an integer `processID`, an integer `remainingTime`, and a `ProcessNode*` `next` pointer.
- Write a standalone function `executeCycle(ProcessNode*& tail, int timeQuantum)`.
- This function must traverse the circular list starting from the head (`tail->next`). For each node, subtract the `timeQuantum` from its `remainingTime`.
- If a node's `remainingTime` drops to 0 or below, the process is complete. You must safely detach it from the circular linked list, re-wire the surrounding pointers to close the gap, and delete the completed node.
- The function should complete exactly one full lap of the circle and return. Account for the edge case where the tail itself finishes execution and needs to be deleted and reassigned.

The Fault Isolation Protocol (Advanced Circular Deletion)

Scenario: A distributed server cluster is infected with self-replicating malware. To isolate the fault and find the "patient zero" node, the security system initiates a systematic shutdown protocol in a ring topology. Starting from the first server, it skips a specific number of active servers and forcefully shuts down the next one, continuing in a circle until only one server remains untouched.

(Note: This is a network-based variation of the classic Josephus Problem).

Tasks:

- Write a standalone function `int findSurvivor(NetworkNode*& tail, int k)`.
- The function takes the tail of a dynamically allocated single circular linked list of servers, and an integer `k`.
- Starting from the head, you must traverse the circle, skip `k-1` nodes, and delete the `k`-th node.
- You must carefully rewire the circular list around the deleted node and continue the count from the immediately subsequent node.
- Repeat this process until only one single node points to itself. Return the `nodeID` of the final surviving node, and leave it safely allocated on the heap.

Reverse Nodes in K-Group (Linear Linked List)

Scenario: You are reverse-engineering a captured data stream from a compromised database. The data packets were sent sequentially as a standard single linked list (NOT circular). To obfuscate the data, the attacker used a rolling cipher that physically reversed the order of the nodes in precise, localized chunks of size `k`. To decode the stream, you must write an algorithm that perfectly undoes this chunked reversal.

Tasks:

- Write a standalone function `PacketNode* reverseKGroup(PacketNode* head, int k)`.
- The function must reverse the nodes of the single linked list exactly `k` nodes at a time, and return the pointer to the new head of the modified list.
- **Strict Constraint 1:** If the total number of remaining nodes at the end of the list is not a perfect multiple of `k`, those leftover nodes **MUST** remain in their original order. They cannot be reversed.
- **Strict Constraint 2:** You may not simply swap or alter the integer data inside the nodes. You must physically rewire the raw next pointers.
- **Strict Constraint 3:** You must solve this using exactly $O(1)$ extra memory space. You are strictly forbidden from placing the nodes into an array or a secondary data structure to reverse them. All logic must be handled via in-place pointer manipulation.

(Example: If the list is 1 -> 2 -> 3 -> 4 -> 5, and $k = 2$, the function must return the list wired as 2 -> 1 -> 4 -> 3 -> 5).